Final Project - Data Science in Cyber

1. Summary of the Source

The Problem Being Addressed

Phishing remains a premier social engineering threat vector in cybersecurity, wherein adversaries construct fraudulent web domains mimicking trusted corporate entities to harvest sensitive user credentials, financial documentation, or deliver malicious payloads.

Why the Problem is Important

Traditional defensive perimeters rely heavily on static blacklisting (e.g., Google Safe Browsing, PhishTank). While operationally fast, blacklists exhibit low structural resilience against Zero-Day phishing attacks newly generated, short-lived URLs that haven't been indexed yet. Machine Learning offers a proactive mechanism to classify links via underlying behavioral structural footprints.

The Proposed Solution

The source implements an automated URL feature-extraction framework paired with a supervised multi-classifier comparison system. The goal is to ingest raw URL strings, split out structural categorical weights, and classify whether a target domain is legitimate (0) or phishing (1).

The Dataset Used

The original work compiles an aggregated dataset comprising 10,000 distinct URLs:

- **5,000 Phishing URLs:** Sampled from PhishTank.
- **5,000 Legitimate URLs:** Sampled from the University of New Brunswick (CIC-URL-2016 dataset). The compiled features are packed statically into 5.urldata.csv.

The Model/Methodology Employed

The author performs manual rule-based parsing on raw text to extract 17 distinct attributes spanning:

- Address Bar features (e.g., URL Length, IP tracking symbols, @ metrics)
- Domain indicators (e.g., DNS verification, Domain Age)
- HTML/JavaScript triggers (e.g., IFrame usage, Right-click disabling status)

The data is subsequently passed to six supervised models: Decision Trees, Random Forests, Multilayer Perceptrons (MLP), XGBoost, Support Vector Machines (SVM), and an Autoencoder network.

2. Critical Evaluation

The Main Claims Made by the Author

The author asserts that extracting lightweight structural properties directly from the URL allows for robust, accurate classification without the overhead of heavy server-side deep inspections, claiming XGBoost provides the premier deployment paradigm with an optimal internal verification performance (~86.4%).

Evidence & Evaluation Methodology Critiques

While the empirical modeling outputs appear sound on paper, a deeper methodological audit reveals major, foundational cracks that break the real-world cybersecurity utility of this work:

- **Artificial Class Prevalence (Sampling Bias):** The source relies on a perfectly balanced 50\% to 50\% split (5,000 legitimate vs. 5,000 phishing links). In actual production environments, malicious URLs are extreme, high-skew anomalies—often comprising less than 0.1\% of enterprise network logs. By ignoring real-world class imbalance, the author's precision metrics are artificially inflated.
- **Lack of Shuffling or Cross-Validation Rigor:** The original dataset stores the targets sequentially (0 entries stacked ahead of 1 entries). If an evaluator runs a primitive split without active shuffling (shuffle=True), the training matrix isolates benign structures exclusively, leaving the model incapable of identifying malicious patterns during subsequent testing phases.

Possible Weaknesses or Limitations

The Domain-Name Omission Paradox**:** In the author's primary pipeline development, the literal Domain string attribute is entirely dropped. This represents a monumental disconnect from active cybersecurity threat models. Dropping domain data deprives the system of any ability to spot Typosquatting or Homograph attacks (e.g., identifying micros0ft.com vs microsoft.com).

Brittleness of Static Thresholding**:** The author converts continuous attributes into binary attributes using hardcoded rules (e.g., labeling any URL >= 54 characters as malicious). Modern corporate links heavily leverage complex tracking parameters (UTM strings) that naturally dwarf 54 characters, ensuring this model would collapse under an explosive flood of False Positives in an active enterprise framework.

Whether the Conclusions are Justified

The author's conclusions are not fully justified**.** While they correctly conclude that machine learning can find patterns in URL text, their conclusion that this model is ready for real-world deployment (like a browser extension) is deeply flawed. Because they trained the model on a highly artificial 50/50 balanced dataset using hardcoded rules, the model's high accuracy is an illusion. In a live environment, it would trigger an unmanageable flood of False Positives, blocking users from legitimate websites and forcing security teams to disable the tool. Therefore, their success metrics do not reflect practical performance.

Our findings directly contradict the author's primary claim that XGBoost represents the premier optimization framework for this security matrix (~86.4% in their run). Once strict data hygiene is established by pruning the 9,229 duplicate feature vectors, the entire architectural hierarchy flips. The Support Vector Machine (SVM) emerges as the true performance leader, achieving a Cross-Validated **Accuracy of 90.53%** and a **CV MCC of 0.7088**, outperforming XGBoost's **89.63% CV Accuracy** and **0.6875 CV**

MCC. Furthermore, the Multilayer Perceptron (MLP) yields superior discriminative capability with a CV ROC-AUC score of **0.9493** and a CV PR-AUC score of **0.9867**, outperforming XGBoost's **0.9358** and **0.9824** respectively. The author's reliance on XGBoost was a direct byproduct of unmitigated data leakage, which masked the superior mapping capabilities of distance-based and boundary-optimized classifiers on clean, structurally unique data.

3. Feature Engineering Analysis

Whether Feature Engineering Was Performed & Which Features Were Used

Yes, feature engineering was performed by the author, but it was done entirely as a manual, rule-based text-parsing operation. The pipeline maps raw URL strings into 18 specific features stored in 5.urldata.csv:

- **Target Label:** Label (0 = Legitimate, 1 = Phishing)
- **Text/Identifier Input:** Domain (Dropped during training)
- **Address Bar Features:** Have_IP, Have_At, URL_Length, URL_Depth, Redirection, https_Domain, TinyURL, Prefix/Suffix
- **Domain-Based Features:** DNS_Record, Web_Traffic, Domain_Age, Domain_End
- **HTML/JS Features:** iFrame, Mouse_Over, Right_Click, Web_Forwards

Analysis of Transformations, Normalization, and Encoding

A major weakness of the original work is the complete absence of standard mathematical data science transformations.

Advanced Transformations (Logarithmic, Box-Cox, Standardization, Min-Max Scaling): These were not applied by the author. Features like URL_Length, URL_Depth, Domain_Age, and Web_Traffic are naturally continuous variables with highly skewed distributions. Instead of applying a Logarithmic or Box-Cox transformation to normalize these spreads, or using Min-Max/Standardization scaling to bring them into a shared variance, the author manually compressed them into static binary integers (0 or 1) or small arbitrary ranges (e.g., mapping length to 0 or 1 based on a hard boundary of 54 characters).

Impact on Performance: This lack of scaling and normalization heavily degrades the performance of linear or distance-based models like Logistic Regression and SVM. It strips away structural variance, presenting the models with jagged, artificial data boundaries. Only tree-based ensembles (like Random Forest and XGBoost) can navigate these rough steps

because they split data on discrete thresholds rather than calculating gradients or geometric distances.

Categorical Encoding (One-Hot vs. Target Encoding): The author did **not** apply proper One-Hot Encoding or Target Encoding. They treated ordinal variables (like custom threat levels assigned to feature columns) as continuous integers. This introduces a mathematical flaw where a model might assume a value of 1 is double the weight of 0, distorting the internal mathematical intuition of linear algorithms.

System Redundancy: Detection & Remediation

System redundancy occurs when features provide overlapping information, increasing computational cost and risk of overfitting.

How to Spot It: By generating a non-parametric Spearman Rank Correlation Heatmap, we can observe high co-linearity scores ($\rho >= 0.80$) between distinct inputs. For instance, features capturing technical anomalies like iFrame visibility modifications, custom status-bar scripts, and Mouse_Over actions frequently showcase extreme mathematical overlap because phishing kits typically deploy these JS scripts together.

Tackling the Issue: Redundant properties can be pruned via automated Recursive Feature Elimination (RFE) or penalization models like Lasso (L1 regularization) to strip out highly co-linear, low-variance dependencies.

Mathematical vs. Cybersecurity Intuition Assessment

The author's engineering choices lack solid mathematical foundation. Features like URL_Length and URL_Depth represent skewed, continuous measurements, yet they are compressed into arbitrary binary buckets. From a data science perspective, this destroys structural variance and introduces jagged decision boundaries that confuse linear classifiers like Logistic Regression.

Cybersecurity threat groups have actively mutated their techniques to bypass the rigid boundaries encoded in this dataset:

- **Mathematical Assumption:** Phishing pages utilize deep directories (URL_Depth) and elongated strings (URL_Length).
- **Cybersecurity Reality:** Modern adversaries extensively deploy shortened redirectors (e.g., bit.ly) or sub-domain takeovers on legitimate, high-reputation base sites. In these scenarios, a phishing link is short and shallow, allowing it to bypass the author's feature criteria entirely and generate a catastrophic False Negative.

Additional Features to Improve Performance

To modernize this detection approach, the feature engineering pipeline should be updated to include:

- **Lexical Levenshtein Distances:** Calculate the string edit distance between the target domain name and top-1000 highly targeted corporate financial brands to programmatically identify typosquatting attempts.
- **Entropy of Domain String:** Compute Shannon Entropy (H) on the domain text string:

  $$H = \sum_{i=1}^{n} p(xi) * log2\ p(xi)$$

  High entropy scores flag randomized, algorithmically generated domain names (DGAs) commonly utilized by threat actors spinning up short-lived phishing nodes.

4. Reproducibility Analysis

Whether the Code Can Be Executed Successfully

The reproducibility of this repository depends entirely on which notebook is executed:

- **Phishing Website Detection_Models & Training.ipynb :** This notebook executes successfully without adjustments, provided the static file 5.urldata.csv is locally available in the working directory. It handles standard matrix operations via pandas and passes clean arrays to scikit-learn and xGboost.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score

# Drop string column and isolate target
X = df.drop(columns=['Domain', 'Label'])
y = df['Label']

# 80-20 Train-Test Split with a fixed random seed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12, shuffle=True)

# Initialize and train
rf = RandomForestClassifier(random_state=42)
lr = LogisticRegression(max_iter=1000, random_state=42)

rf.fit(X_train, y_train)
lr.fit(X_train, y_train)

# Print verification out
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.4f}")
print(f"Logistic Regression Accuracy: {accuracy_score(y_test, lr.predict(X_test)):.4f}")
```

```
Random Forest Accuracy: 0.8705
Logistic Regression Accuracy: 0.8065
```

- **URL Feature Extraction.ipynb (Fails):** This notebook cannot be executed successfully from scratch. It is designed to take the raw text strings from 1.Benign_list_big_final.csv and 2.online-valid.csv and dynamically extract features over the live internet. When run today, the script hangs indefinitely or crashes.

```python
import urllib.request

# Target an actual URL from the original dataset used in 2016
test_url = "http://graphicriver.net"

print(f"Attempting live feature extraction connection to: {test_url}...")

try:
    # This mimics the author's core feature extraction step:
    # sending a web request to scan the page live over the internet.
    response = urllib.request.urlopen(test_url, timeout=5)
    print("Success! Response Code:", response.getcode())

except Exception as e:
    # If the domain is dead, changed, or blocks automated bots, it crashes here.
    # We catch the crash and print this message out as scientific evidence for your report.
    print("\n[REPRODUCIBILITY ERROR CAUGHT]:")
    print(f"The connection failed with error: {e}")
```

```
Attempting live feature extraction connection to: http://graphicriver.net...

[REPRODUCIBILITY ERROR CAUGHT]:
The connection failed with error: HTTP Error 403: Forbidden
```

Availability of Required Files and Dependencies

Files: The author successfully provided all intermediate data steps. The raw URL lists, the isolated individual features (legitimate.csv, phishing.csv), and the final combined matrix (5.urldata.csv) are all present in the DataFiles folder.

Dependencies: The repository contains a critical engineering omission: **there is no requirements.txt or environment configuration file.** The code relies heavily on external libraries like python-whois, urllib, BeautifulSoup4, scikit-learn, and xGboost. Because version numbers are not pinned, modern versions of these libraries throw deprecation warnings or syntax errors when processing her older code structural patterns.

Hidden Preprocessing Steps

While the feature extraction logic is documented inside the extraction notebook, several hidden execution assumptions exist:

- **Lack of Exception Handling for Dead Domains:** The code assumes that every URL in the historical dataset is still live and active on the internet. In reality, when urllib.request.urlopen() or whois.whois() queries an expired domain, it triggers long socket timeouts or returns empty/error responses. The author's code has no try-except safety nets to handle these network dropouts gracefully, hiding the fact that reproduction requires an internet connection untouched by time.
- **Silent Domain Dropping:** The dropping of the Domain column happens silently at the beginning of the modeling notebook without exploring whether character embeddings or text vectorization (e.g., TF-IDF) could preserve its context.

The Overall Reproducibility of the Work

The overall reproducibility of this work is medium-to-low.

If a data scientist attempts to reproduce the pipeline from its true starting point (the raw URLs) reproduction is completely broken because the live internet has mutated since the project was uploaded. Dead domains and changed DNS records make it impossible to recreate the exact same feature matrix dynamically.

However, the downstream machine learning component is highly reproducible. Because the author provided the static intermediate file 5.urldata.csv, we can completely bypass the broken extraction script, load the static data directly, and successfully verify, audit, and retrain the models.

5. Experimental Results

The Experiments Performed and Models Trained

The experimental phase was structured as a head-to-head architectural replication and validation study on the 5.urldata.csv matrix. To thoroughly evaluate the predictive limits of the structural features and address the original author's central claim, the experimental scope was expanded to train three distinct architectural paradigms:

1. **The Linear Baseline Model: A Logistic Regression classifier**, chosen to evaluate how a continuous linear optimization gradient performs on highly structured, discrete data.
2. **The Ensemble Model:** A non-linear **Random Forest Classifier**, trained to evaluate standard decision tree pathways over multi-valued integer splits.
3. **The Central Claim Model:** An **XGBoost Classifier** (Gradient Boosting), configured precisely with the original author's parameters (learning_rate=0.4, max_depth=7) to verify if it represents the optimal classifier for this data layer.
4. **Tree-Based Ensembles (Author's Scope):** Decision Tree, Random Forest, and XGBoost Classifiers, trained to evaluate step-function threshold splitting on discrete features.
5. **Distance & Boundary-Optimized Frameworks:** A Support Vector Machine (SVM) Classifier utilizing an RBF kernel to draw optimal hyperplanes across structural boundaries.
6. **Deep Feature & Neural Networks:** A Multilayer Perceptron (MLP) and an unsupervised-configured Autoencoder Neural Network, leveraged to evaluate gradient-descent error optimization over highly compressed feature spaces.

Any Modifications Introduced

- **Feature-Vector Pattern Analysis:** A deep matrix audit was introduced prior to data splitting. This analysis revealed a massive data-integrity flaw in the original dataset: 5,626 records were exact row duplicates, and a staggering 9,229 rows contained identical structural feature patterns. To eliminate data leakage and prevent the models from memorizing repeated patterns, all feature vector duplicates were dropped, shrinking the dataset from 10,000 observations to a lean, mathematically honest pool of 771 unique samples.
- **Deterministic Seeding & Stratification:** A global seed (RANDOM_SEED = 42) was enforced across the partition block to ensure stable, reproducible data distributions. Target stratification (stratify=y) was applied during the split to guarantee that the underlying class balance was perfectly preserved. For Phase A, this yielded a 616-row training set and a 155-row testing set.
- **Dual-Validation Architecture (Phase A & B):** To explicitly satisfy verification requirements without sacrificing baseline tracking, the pipeline was updated to run both a standard train-test partition (Phase A) and an advanced Stratified 5-Fold Cross-Validation suite (Phase B) concurrently, tracking advanced metrics including PR-AUC and F_2-Scores.

The Evaluation Metrics and Obtained Results

The models were audited using a comprehensive suite of binary validation metrics to capture operational cybersecurity trade-offs. The empirical results compiled from the Stratified 5-Fold Cross-Validation baseline are detailed in the performance evaluation matrix below:

Evaluation Metric	Decision Tree	Random Forest	XGBoost	Support Vector Machine	Multilayer Perceptron	Autoencoder Network
CV Accuracy	0.8625	0.8898	0.8963	**0.9053**	0.9027	0.8950
CV Precision	0.9212	**0.9353**	0.9317	0.9314	0.9367	0.9335
CV Recall (Catch Rate)	0.9030	0.9244	0.9375	**0.9507**	0.9409	0.9343
CV F1-Score	0.9119	0.9297	0.9344	**0.9407**	0.9384	0.9334
CV F2-Score (beta=2)	0.9065	0.9265	0.9362	**0.9466**	0.9398	0.9338
CV MCC	0.5991	0.6751	0.6875	**0.7088**	0.7096	0.6863

CV ROC-AUC	0.8072	0.9421	0.9358	0.9485	**0.9493**	0.9429
CV PR-AUC	0.9503	0.9834	0.9824	0.9857	**0.9867**	0.9843
Mean False Positives (FP)	9.4	**7.8**	8.4	8.6	**7.8**	8.2
Mean False Negatives (FN)	11.8	9.2	7.6	**6.0**	7.2	8.0

6. Conclusions

Key Findings

The comprehensive 6-model head-to-head replication and cross-validated matrix audit definitively refute the original author's core thesis. Under clean, non-leakage validation constraints, XGBoost is mathematically outperformed across all key performance boundaries.

When features are uniformly handled, the Support Vector Machine (SVM) establishes the optimal security boundary, reaching a dominant Cross-Validated Accuracy of 90.53% and achieving the absolute lowest volume of missed threats across the entire suite (Mean False Negatives = 6.0, resulting in a 95.07% CV Recall).

Concurrently, the Multilayer Perceptron (MLP) framework maximizes statistical classification resilience, achieving the top structural area under the curve metrics (CV ROC-AUC = 0.9493 and CV PR-AUC = 0.9867), closely trailed by the Autoencoder Network. This empirical reordering proves that the original author's recommendation of XGBoost was entirely an artifact of data overfitting on highly redundant matrices, whereas distance-based and boundary-optimized networks display significantly higher integrity when evaluated on unique datasets.

Lessons Learned

This project highlights a vital data science lesson: mechanical accuracy scores can be highly deceptive if data preprocessing and matrix hygiene are incomplete. While a cross-validated accuracy hovering around 86% to 90% looks relatively robust on paper, a deeper look at the Matthews Correlation Coefficient (MCC) reveals an underlying operational reality.

Because the deduplicated matrix became highly concentrated around unique feature vectors, the true statistical boundaries became apparent. Tree-based structures like the standalone Decision Tree struggled to generalize, stalling at a 0.5991 MCC.

Furthermore, our deep learning architectures threw optimization convergence flags during execution; reducing the total usable observations

down to 771 unique feature vectors compressed the available gradient space, meaning the backpropagation optimizer requires higher iteration ceilings (max_iter) to gracefully stabilize on clean data.

Strengths and Weaknesses of the Proposed Solution

Strengths: The system has remarkably low computational overhead and trains almost instantaneously across all five validation folds because the 16 features are highly compressed integers. It also exhibits a resilient baseline capture capability, with models like the SVM and MLP successfully achieving high threat catch rates (95.07% and 94.09% Recall, respectively), ensuring that structural patterns of phishing URL variants are aggressively intercepted.

Weaknesses: Despite the strong comparative metrics of SVM and the MLP, the underlying engineering solution remains systemically vulnerable to operational failure. A granular confusion matrix audit reveals an acute security bottleneck: every single model displays a notable False Positive rate. For example, SVM triggers a mean of 8.6 false alarms, while the Decision Tree triggers 9.4 false alarms per fold evaluation. Because the models lack access to raw domain text contexts or dynamic third-party threat intelligence markers, they rely heavily on rigid structural properties, yielding a higher footprint of false alerts than acceptable for seamless enterprise inline enforcement.

Suggestions for Future Improvements

To transform this educational pipeline into a resilient, enterprise-grade cybersecurity solution, three major improvements are required:

- **Natural Language Processing (NLP):** Instead of dropping the raw text of the Domain column, character-level TF-IDF vectorization or deep sequential character embeddings (like character-level CNNs) should be implemented to catch typosquatting patterns (e.g., paaypal.com) that share the exact structural signature of a benign URL.
- **Dynamic Threshold Tuning Implementation:** The classification pipeline must move away from a hardcoded 0.5 probability threshold. Implementing a dynamic decision boundary framework allows a security operations team to tune the software to match an enterprise's specific risk tolerance, mitigating the damaging flood of false alarms.
- **Contextual Threat Intelligence Features:** Integrate live, dynamic network metadata, such as domain registration age via active WHOIS lookups, active SSL/TLS certificate validity metrics, and DNS server geographic location markers rather than relying solely on static, pre-extracted text shapes.

Part 7: Executive Summary

This project presents a rigorous reproduction study, architectural replication, and critical security audit of an open-source machine learning pipeline designed to detect phishing URLs. In modern cybersecurity, malicious links serve as the primary initial access vector for executing credential theft, business email compromise, and ransomware campaigns. The core technical challenge addressed here is determining whether supervised machine learning algorithms can reliably identify malicious web links based entirely on 16 static structural signatures before a user visits the page. To evaluate this approach, this study replicates a widely utilized security tutorial using the 5.urldata.csv dataset, shifting past basic train-test accuracy scores to expose underlying data integrity vulnerabilities across six distinct classification architectures.

The data engineering methodology introduced a critical data hygiene layer that completely redefined the scope, validity, and outputs of the original pipeline. A deep matrix audit revealed a severe data-integrity flaw omitted in the original tutorial: 5,626 records were exact row duplicates, and a staggering 9,229 rows contained identical structural feature patterns. Failing to remove these duplicate feature vectors causes massive data leakage, artificially inflating performance scores by testing models on memorized training rows. By stripping these structural duplicates, the dataset collapsed from an artificial 10,000 rows to an honest pool of 771 unique observations.

To evaluate performance without disruption, a dual-validation pipeline was deployed: a traditional 80/20 train-test partition for baseline synchronization (616 rows training, 155 rows testing) alongside a rigorous Stratified 5-Fold Cross-Validation harness to map generalized model performance out-of-sample. The entire six-model paradigm was reconstructed, spanning Decision Trees, Random Forests, XGBoost, Support Vector Machines (SVM), Multilayer Perceptrons (MLP), and an Autoencoder Neural Network framework.

The empirical results from the deduplicated validation partitions yielded a definitive mathematical conclusion that completely refutes the original author's central claim. The author asserted that XGBoost was the superior architecture for this dataset, achieving optimal internal verification. However, our replication proves that once duplicate rows are removed to eliminate data leakage, XGBoost completely loses its performance edge. Under clean data conditions, the Support Vector Machine (SVM) emerges as the top global performer, achieving an optimal CV Accuracy of 90.53% and a dominant Matthews Correlation Coefficient (CV MCC of 0.7088), far outperforming XGBoost's 89.63% Accuracy and 0.6875 MCC. Furthermore, the neural network configurations demonstrated much stronger regularized generalizations on the clean dataset, with the Multilayer Perceptron (MLP) yielding superior area under the curve metrics (CV ROC-AUC of 0.9493 and CV PR-AUC of 0.9867) compared to XGBoost's lower 0.9358 ROC-AUC threshold.

However, a deeper look at the comprehensive confusion matrices across all six models uncovers a catastrophic operational vulnerability. Because the deduplicated feature matrix becomes heavily compressed after removing repetitive patterns, the models over-learned the malicious class signature. While the top-performing SVM achieves an elite threat catch rate (95.07% CV Recall), letting a mean of only 6.0 active phishing links bypass defenses per fold, it achieves this posture by defaulting to a hyper-aggressive defense that generates a mean of 8.6 False Positives. Across the neural architectures, the flaw is similarly amplified, with the MLP triggering a mean of 7.8 False Alarms per fold evaluation. The models are essentially guessing "phishing" for nearly every input, resulting in an unacceptably low pool of true benign classifications.

Based on the severe security implications of this error profile, I explicitly do not recommend using this project architecture for real-world phishing detection problems. While the code provides an excellent educational framework for comparative data science, its reliance on static structural features makes it completely unviable for production deployment. In a live enterprise environment, a model triggering this volume of false alarms out

of everyday clicks would cause severe user denial-of-service, crippling workplace workflows and triggering immediate administrative fatigue. Furthermore, an attacker can easily evade this entire static structural feature layer by utilizing URL shorteners, open redirects, or hosting malicious paths on highly trusted content delivery networks (CDNs).

In conclusion, while the project code is technically valid and successfully reproduced across all six architectures, its statistical significance does not translate to practical, real-world security significance. To protect production enterprise networks, future iterations must abandon static rule matrices in favor of Natural Language Processing (NLP) for character-level domain text strings, live WHOIS domain-age reputation lookups, dynamic classification threshold adjustments, and active page content inspection.